

CS 440 Assignment 1 Report

Team Details

- Reuben Geronimo (`rg1090`)
- Orland Geronimo (`ogg9`)

Part 1a - Why the first move is east in Figure 8

The agent plans with the freespace assumption, so unknown cells are treated as unblocked until observed otherwise. On the first search in Figure 8, moving east is on a shortest presumed-unblocked path to the target.

If multiple first moves have the same path cost (for example east and north), the implementation's tie-breaking and neighbor-processing order pick one deterministically. In our implementation, that first move is east.

Part 1b - Why the agent always terminates in finite time; move upper bound

Let U be the number of truly unblocked cells in the finite grid.

1. **Why it terminates in finite time**

- Each planning cycle is finite.
- After each cycle, the agent either reaches the goal, proves no presumed path exists, or learns new blockage info.
- Because the grid is finite, the amount of new information is finite.
- So the whole process must finish in finite time.

2. **Move bound**

- In one cycle, the agent follows a shortest presumed-unblocked path, so the path length is at most $U-1$.
- The number of meaningful replans is finite and bounded in finite grids.
- A coarse bound is:
$$\text{moves} \leq U \cdot (U-1) < U^2.$$

So the total number of moves until success/failure is upper-bounded by the number of unblocked cells squared.

Part 2 - Effects of tie-breaking in Repeated Forward A*

We compared two versions:

- max_g : break f ties toward larger g
- min_g : break f ties toward smaller g

All runs were on the same 50 mazes (101×101 , seed 42).

Observations

- Found rate: same for both (26/50)
- Expanded nodes:
 - Mean: 8,920 (max_g) vs 201,837 (min_g)
 - Median: 8,790 (max_g) vs 242,449 (min_g)
 - Per-maze: max_g better on 46/50, ties 4/50, min_g better on 0/50
- Runtime:
 - Mean: 76.7 ms (max_g) vs 1,190.4 ms (min_g)
 - Median: 80.9 ms (max_g) vs 1,445.2 ms (min_g)

Explanation

When $f = g + h$ ties happen, choosing larger g tends to move search focus clo

ser to the goal side of the same `f` contour. Choosing smaller `g` keeps more search effort near the start side. That is why `min\_g` expands much more and runs much slower in our experiments.

Part 3 - Repeated Forward A* vs Repeated Backward A*

We compared:

- Forward (`max\_g`)
- Backward (`max\_g`)

Observations

- Found rate: same (`26/50`)
- Expanded nodes:
 - Mean: `8,920` (forward) vs `102,180` (backward)
 - Median: `8,790` (forward) vs `101,170` (backward)
 - Forward better on expansions: `50/50` mazes
- Runtime:
 - Mean: `77.1 ms` (forward) vs `1,325.6 ms` (backward)
 - Median: `82.2 ms` (forward) vs `1,283.9 ms` (backward)
 - Forward better on runtime: `50/50` mazes

Explanation

In this problem, new information is discovered around the agent as it moves. Planning from the current agent state to the goal aligned better with that information pattern than planning from goal back to current state, so forward search was consistently cheaper in our 50-maze set.

Part 4a - Why Manhattan distance is consistent here

Heuristic:

$$h(s) = |x_s - x_g| + |y_s - y_g|.$$

For neighboring cells `s` and `s'` in a 4-direction grid, one step changes Manhattan distance by at most 1, so:

$$h(s) \leq 1 + h(s').$$

Since each move cost is `c(s,s')=1`, this is exactly:

$$h(s) \leq c(s,s') + h(s').$$

So Manhattan distance is consistent.

Part 4b - Why Adaptive A* keeps heuristics admissible/consistent when costs increase

Adaptive A* updates expanded states with:

$$h_{\text{new}}(s) = g(s_{\text{goal}}) - g(s).$$

At update time, that value is a valid lower bound to goal distance for expanded states, so it is admissible. Later, edge costs can only increase (not decrease), which means true shortest distances cannot go down; therefore an admissible lower bound stays admissible.

For consistency: updates come from shortest-path `g` values in that search, which satisfy triangle-inequality-style relations. Non-updated states keep previously consistent values. With only cost increases and these updates, consistency is preserved across searches.

So Adaptive A* keeps initially consistent heuristics both admissible and consistent.

Part 5 - Repeated Forward A* vs Adaptive A*

We compared:

- Forward (`max\_g`)
- Adaptive (`max\_g`)

Observations

- Found rate: same (`26/50`)
- Expanded nodes:
 - Mean: `8,920` (forward) vs `8,549` (adaptive)
 - Median: `8,790` (forward) vs `8,717` (adaptive)
 - Per-maze: adaptive better `34`, ties `14`, worse `2`
- Runtime:
 - Mean: `76.0 ms` (forward) vs `74.6 ms` (adaptive)
 - Median: `81.4 ms` (forward) vs `78.6 ms` (adaptive)

Explanation

Adaptive A* reuses prior search work by raising heuristic values on expanded states:

$$h_{\text{new}}(s) = g(s_{\text{goal}}) - g(s).$$

That usually focuses later replans and lowers expansions. Runtime improves only slightly because overhead/noise can mask small gains.

Part 6 - Statistical significance (how + results)

To test whether differences are real (not just sample noise), we use a paired two-sided sign test across the same 50 mazes.

For each maze:

$$d_i = \text{metric}_{\text{B},i} - \text{metric}_{\text{A},i}.$$

- `B` win if `d\_i < 0`
- `B` loss if `d\_i > 0`
- ties (`d\_i=0`) dropped

Null hypothesis:

$$H_0: P(\text{B win}) = 0.5.$$

Two-sided exact p-values are computed from $\text{Binomial}(n, 0.5)$, where $n = \text{wins} + \text{losses}$.

Expanded-node significance results

- Part 2 (`min\_g` vs `max\_g`): wins=0, losses=46, ties=4
($p \approx 2.84 \times 10^{-14}$)

- Part 3 (`bwd` vs `fwd`): wins=0, losses=50, ties=0
 $(p \approx 1.78 \times 10^{-15})$
- Part 5 (`adaptive` vs `fwd`): wins=34, losses=2, ties=14
 $(p \approx 1.94 \times 10^{-8})$

At $(\alpha=0.05)$, all three expanded-node differences are statistically significant.

Runtime Interpretation

For Part 5 runtime, wins=25 and losses=25 (ties=0), so $(p=1.0)$. That means no statistically significant paired runtime advantage, even though the mean/median runtime is slightly better for adaptive.